

Data Structure and Algorithm

June Lazarus

2026 Spring

目录

前言	3
1 引言	4
1.1 什么是 DSA?	4
1.2 Python 中的面向对象编程	4
2 复杂度分析	7
2.1 Big-O 记号	7
2.2 Python 操作复杂度	7
3 线性数据结构	9
3.1 顺序存储 vs. 链式存储	9
3.2 链表	9
3.2.1 节点定义	9
3.2.2 核心操作	10
3.3 栈	11
3.3.1 单调栈	11
3.4 队列与 BFS	12
3.5 哈希表	12
4 倒排索引	14
4.1 倒排索引 (Inverted Index)	14
5 位运算与前缀和	15
5.1 位运算	15
5.1.1 常用位运算技巧	15

目录	3
5.1.2 状态压缩	16
5.2 前缀和	16
6 树	18
6.1 树的术语	18
6.1.1 卡特兰数 (Catalan Numbers)	18
6.2 二叉树	19
6.3 树的遍历	19
6.3.1 前序遍历 (根 $\rightarrow$ 左 $\rightarrow$ 右)	19
6.3.2 中序遍历 (左 $\rightarrow$ 根 $\rightarrow$ 右)	19
6.3.3 后序遍历 (左 $\rightarrow$ 右 $\rightarrow$ 根)	20
6.3.4 层序遍历 (BFS)	20
6.4 二叉搜索树 (BST)	21
6.5 堆 (Heap)	23
6.6 哈夫曼编码 (Huffman Coding)	24
6.6.1 算法步骤	25
6.7 字典树 (Trie)	26
6.8 并查集 (Disjoint Set Union)	27
7 图	28
7.1 术语与定义	28
7.2 图的表示	28
7.2.1 邻接矩阵	28
7.2.2 邻接表	29
7.3 图的遍历	29
7.3.1 BFS (广度优先搜索)	29
7.3.2 DFS (深度优先搜索)	30
7.4 拓扑排序	31
7.4.1 Kahn 算法 (BFS 实现)	31
7.5 AOE 网与关键路径	32
7.5.1 AOE 网 (Activity On Edge)	32
7.5.2 关键路径 (Critical Path)	32
7.5.3 核心量	32
7.5.4 算法步骤	33

7.6	最短路径	34
7.6.1	Dijkstra 算法	34
7.6.2	Bellman-Ford 算法	35
7.6.3	Floyd-Warshall 算法	35
7.7	最小生成树 (MST)	36
7.7.1	Kruskal 算法	36
7.7.2	Prim 算法	37
7.8	强连通分量 (SCC)	37
7.8.1	Tarjan 算法	38
A	常用算法模式	40
A.1	双指针	40
A.2	滑动窗口	41
A.3	动态规划	42
A.4	回溯	42
A.5	二分查找	43

前言

本文档旨在更好地查找、学习和使用本学期所学的算法。全书涵盖数据结构与算法（DSA）的核心内容，从线性结构到树与图的高级算法。

全书共分为七章：

- 第一章：引言与面向对象编程
- 第二章：复杂度分析（Big-O 记号）
- 第三章：线性数据结构
- 第四章：倒排索引
- 第五章：位运算与前缀和
- 第六章：树结构
- 第七章：图算法

每章包含核心概念、Python 实现，以及选自 LeetCode、OpenJudge、Codeforces 和洛谷的精选例题。

June Lazarus
2026 Spring

Chapter 1

引言

1.1 什么是 DSA?

数据结构与算法（DSA）结合了两个密切相关的领域：

- **数据结构**：组织和存储数据的方式，以便高效访问和修改。例如：数组、链表、树、图。
- **算法**：解决问题的分步过程。例如：排序、搜索、图遍历。

数据结构的选择直接影响算法性能。选对数据结构可以将时间复杂度从 $O(n^2)$ 降低到 $O(n)$ 。

1.2 Python 中的面向对象编程

OOP 是我们实现数据结构所使用的编程范式。核心概念：

- **封装**：数据和方法被绑定在类中。
- **继承**：类可以从父类继承属性和方法。
- **对象**：类的实例，每个对象都有独立的状态。

```
1 class DisjointSet:
2     """ 并查集 —— 管理不相交集合的数据结构。 """
3     def __init__(self, n):
```

```

4         self.rank = [1] * n
5         self.parent = [i for i in range(n)]
6
7     def find(self, x):
8         """ 查找元素  $x$  所属集合的代表元，含路径压缩。 """
9         if self.parent[x] != x:
10             self.parent[x] = self.find(self.parent[x])
11         return self.parent[x]
12
13     def union(self, x, y):
14         """ 合并  $x$  和  $y$  所在的集合，按秩合并。 """
15         x_root = self.find(x)
16         y_root = self.find(y)
17         if x_root == y_root:
18             return
19         if self.rank[x_root] < self.rank[y_root]:
20             self.parent[x_root] = y_root
21         elif self.rank[x_root] > self.rank[y_root]:
22             self.parent[y_root] = x_root
23         else:
24             self.parent[y_root] = x_root
25             self.rank[x_root] += 1

```

例题：最小栈 (LeetCode 155)

设计一个支持 push、pop、top 以及常数时间内检索最小元素的栈。

```

1 class MinStack:
2     """ 支持  $O(1)$  时间获取最小值的栈。 """
3     def __init__(self):
4         self.stack = []
5         self.min_stack = []
6
7     def push(self, val: int) -> None:
8         self.stack.append(val)
9         if not self.min_stack or val <= self.min_stack[-1]:
10             self.min_stack.append(val)
11
12     def pop(self) -> None:
13         if self.stack.pop() == self.min_stack[-1]:
14             self.min_stack.pop()
15
16     def top(self) -> int:
17         return self.stack[-1]
18
19     def getMin(self) -> int:
20         return self.min_stack[-1]

```

关键思路：使用辅助栈跟踪每个层级的最小值。仅在新值 $\leq$ 栈顶时才压入辅助栈。

Chapter 2

复杂度分析

2.1 Big-O 记号

渐近记号描述算法运行时间或空间随输入规模 n 增长的趋势：

- $O(g(n))$ ：上界（最坏情况）。
- $\Omega(g(n))$ ：下界（最好情况）。
- $\Theta(g(n))$ ：紧确界（平均情况）。

记号	名称	示例
$O(1)$	常数阶	数组索引
$O(\log n)$	对数阶	二分查找
$O(n)$	线性阶	数组遍历
$O(n \log n)$	线性对数阶	归并排序
$O(n^2)$	平方阶	冒泡排序
$O(2^n)$	指数阶	递归求斐波那契

表 2.1: 常见复杂度量级

2.2 Python 操作复杂度

- `list.append(x)`：均摊 $O(1)$ 。

- `list.insert(0, x)`: $O(n)$ 。
- `dict[key]`、`set.add(x)`: 均摊 $O(1)$ 。
- `x in list`: $O(n)$; `x in set`: $O(1)$ 。

例题：两数之和 (LeetCode 1)

暴力法: $O(n^2)$ 嵌套循环。优化: $O(n)$ 用哈希表。

```
1 def twoSum(nums, target):
2     """ 在数组中找到和为目标值的两个数的索引。 """
3     seen = {}
4     for i, num in enumerate(nums):
5         if target - num in seen:
6             return [seen[target - num], i]
7     seen[num] = i
```

Chapter 3

线性数据结构

3.1 顺序存储 vs. 链式存储

特性	数组	链表
内存布局	连续存储	分散节点
随机访问	$O(1)$	$O(n)$
插入/删除(已知位置)	$O(n)$ (需移动)	$O(1)$
空间开销	预分配可能浪费	每个节点额外存储指针

表 3.1: 数组与链表对比

3.2 链表

3.2.1 节点定义

```
1 class ListNode:
2     """ 单链表节点。 """
3     def __init__(self, val=0, next=None):
4         self.val = val
5         self.next = next
```

3.2.2 核心操作

```

1 def reverseList(head: ListNode) -> ListNode:
2     """ 反转单链表。 """
3     prev, cur = None, head
4     while cur:
5         nxt = cur.next
6         cur.next = prev
7         prev, cur = cur, nxt
8     return prev
9
10 def hasCycle(head: ListNode) -> bool:
11     """ 使用快慢指针检测链表是否有环。 """
12     slow = fast = head
13     while fast and fast.next:
14         slow = slow.next
15         fast = fast.next.next
16     if slow == fast:
17         return True
18     return False
19
20 def getIntersectionNode(headA, headB):
21     """ 寻找两个链表的交点。 """
22     a, b = headA, headB
23     while a != b:
24         a = a.next if a else headB
25         b = b.next if b else headA
26     return a

```

例题：LRU 缓存 (LeetCode 146)

将哈希表与双向链表结合，实现 $O(1)$ 时间的 get 和 put:

```

1 class DLinkedNode:
2     """ 双向链表节点。 """
3     def __init__(self, key=0, val=0):
4         self.key, self.val = key, val
5         self.prev = self.next = None
6
7 class LRUCache:
8     """ LRU 缓存：最近最少使用淘汰策略。 """
9     def __init__(self, capacity: int):
10         self.cap = capacity
11         self.cache = {}
12         self.head, self.tail = DLinkedNode(), DLinkedNode()
13         self.head.next, self.tail.prev = self.tail, self.head
14
15     def _remove(self, node):

```

```

16         """ 从链表中移除节点。 """
17         node.prev.next, node.next.prev = node.next, node.prev
18
19     def _add_to_head(self, node):
20         """ 将节点添加到链表头部。 """
21         node.next = self.head.next
22         node.prev = self.head
23         self.head.next.prev = node
24         self.head.next = node
25
26     def get(self, key: int) -> int:
27         if key not in self.cache:
28             return -1
29         node = self.cache[key]
30         self._remove(node)
31         self._add_to_head(node)
32         return node.val
33
34     def put(self, key: int, val: int) -> None:
35         if key in self.cache:
36             self._remove(self.cache[key])
37         node = DLinkedNode(key, val)
38         self.cache[key] = node
39         self._add_to_head(node)
40         if len(self.cache) > self.cap:
41             rm = self.tail.prev
42             self._remove(rm)
43             del self.cache[rm.key]

```

3.3 栈

栈是一种后进先出（LIFO）的数据结构。

3.3.1 单调栈

单调栈维护元素的递增或递减顺序，常用于”下一个更大元素”类问题：

```

1 def maxSlidingWindow(nums, k):
2     """ 滑动窗口最大值 (LeetCode 239)。 """
3     from collections import deque
4     dq, res = deque(), []
5     for i in range(len(nums)):
6         while dq and nums[dq[-1]] < nums[i]:
7             dq.pop()
8         dq.append(i)
9         if dq[0] == i - k:

```

```

10         dq.popleft()
11         if i >= k - 1:
12             res.append(nums[dq[0]])
13     return res

```

练习题：有效的括号 (E20)、字符串解码 (M394)、柱状图中最大的矩形 (T84)。

3.4 队列与 BFS

队列是先进先出 (FIFO)。BFS 使用队列进行按层探索。

例题：腐烂的橘子 (LeetCode 994)

```

1  from collections import deque
2
3  def orangesRotting(grid):
4      """ 返回所有新鲜橘子腐烂所需的最少分钟数。 """
5      m, n = len(grid), len(grid[0])
6      q = deque()
7      fresh = 0
8      for i in range(m):
9          for j in range(n):
10             if grid[i][j] == 2:
11                 q.append((i, j, 0))
12             elif grid[i][j] == 1:
13                 fresh += 1
14
15      max_t = 0
16      dirs = [(1,0), (-1,0), (0,1), (0,-1)]
17      while q:
18          x, y, t = q.popleft()
19          max_t = max(max_t, t)
20          for dx, dy in dirs:
21              nx, ny = x + dx, y + dy
22              if 0 <= nx < m and 0 <= ny < n and grid[nx][ny] == 1:
23                  grid[nx][ny] = 2
24                  fresh -= 1
25                  q.append((nx, ny, t + 1))
26      return max_t if fresh == 0 else -1

```

3.5 哈希表

哈希表在平均情况下提供 $O(1)$ 的操作：Python 中的 `dict` 和 `set`。

例题：和为 K 的子数组 (LeetCode 560)

前缀和 + 哈希表：

```
1 def subarraySum(nums, k):
2     """ 统计和为 k 的连续子数组个数。 """
3     count = cur = 0
4     prefix = {0: 1}
5     for num in nums:
6         cur += num
7         count += prefix.get(cur - k, 0)
8         prefix[cur] = prefix.get(cur, 0) + 1
9     return count
```

Chapter 4

倒排索引

4.1 倒排索引 (Inverted Index)

倒排索引将词条映射到其所在的文档位置——这是搜索引擎背后的核心数据结构。

```
1 from collections import defaultdict
2
3 def buildInvertedIndex(documents):
4     """ 构建倒排索引。
5     参数:
6         documents: 文档列表, 每个文档为字符串。
7     返回:
8         {词语: {文档 ID: [位置列表]}}。
9     """
10    index = defaultdict(lambda: defaultdict(list))
11    for doc_id, doc in enumerate(documents):
12        for pos, word in enumerate(doc.split()):
13            index[word][doc_id].append(pos)
14    return index
```

应用场景: 搜索引擎全文检索、日志分析、关键词高亮。

练习: 倒排索引查询 (OpenJudge 04093)

给定一组文档, 构建倒排索引并支持多关键词联合查询。

Chapter 5

位运算与前缀和

5.1 位运算

位运算直接对整数的二进制位进行操作，执行速度极快。

运算	Python	效果
与	<code>a & b</code>	按位与
或	<code>a b</code>	按位或
异或	<code>a ^ b</code>	按位异或
非	<code>~a</code>	按位取反
左移	<code>a << k</code>	乘以 2^k
右移	<code>a >> k</code>	除以 2^k

表 5.1: 位运算操作

5.1.1 常用位运算技巧

```
1 def lowbit(x: int) -> int:
2     """ 提取最低位的 1。例如: lowbit(12) = lowbit(0b1100) = 4。 """
3     return x & (-x)
4
5 def isPowerOfTwo(x: int) -> bool:
6     """ 判断 x 是否为 2 的幂。 """
7     return x > 0 and (x & (x - 1)) == 0
8
```

```

9 def countBits(x: int) -> int:
10     """ 统计二进制中 1 的个数 (汉明重量)。 """
11     return x.bit_count()
12
13 def setBit(x: int, k: int) -> int:
14     """ 设置第 k 位 (从最低位 0 开始计数)。 """
15     return x | (1 << k)
16
17 def clearBit(x: int, k: int) -> int:
18     """ 清除第 k 位。 """
19     return x & ~(1 << k)
20
21 def toggleBit(x: int, k: int) -> int:
22     """ 翻转第 k 位。 """
23     return x ^ (1 << k)
24
25 def isBitSet(x: int, k: int) -> bool:
26     """ 检查第 k 位是否为 1。 """
27     return (x >> k) & 1

```

5.1.2 状态压缩

可以用比特位紧凑地表示集合 $\{0, 1, \dots, n-1\}$ 的子集:

```

1 def allSubsets(n: int):
2     """ 生成 n 个元素集合的所有子集。 """
3     for mask in range(1 << n):
4         subset = [i for i in range(n) if (mask >> i) & 1]
5         yield subset
6
7 def isSubset(mask: int, sub: int) -> bool:
8     """ 检查 sub 是否为 mask 的子集。 """
9     return (mask & sub) == sub

```

例题：根据二进制中 1 的个数排序 (LeetCode 1356)

```

1 def sortByBits(arr):
2     """ 按二进制中 1 的个数对数组排序，个数相同则按数值排序。 """
3     return sorted(arr, key=lambda x: (x.bit_count(), x))

```

5.2 前缀和

前缀和能够在 $O(n)$ 预处理后，实现 $O(1)$ 的区间和查询：

```
1 class PrefixSum:
2     """ 前缀和 ——  $O(1)$  时间查询任意区间和。 """
3     def __init__(self, nums):
4         self.p = [0]
5         for x in nums:
6             self.p.append(self.p[-1] + x)
7
8     def rangeSum(self, l: int, r: int) -> int:
9         """ 返回 nums[l..r] 的区间和 (闭区间)。 """
10        return self.p[r + 1] - self.p[l]
```

二维前缀和也是常见考点（如 LeetCode 304：二维区域和检索 - 矩阵不可变）。

Chapter 6

树

树用于建模层次关系。它们出现在文件系统、HTML DOM、决策过程等场景中。

6.1 树的术语

- **根节点 (Root):** 最顶端的节点（无父节点）。
- **叶子 (Leaf):** 无子节点的节点。
- **深度 (Depth):** 从根到节点 x 的路径边数。
- **高度 (Height):** 从节点 x 到最深叶子的最长路径边数。
- **二叉树 (Binary Tree):** 每个节点最多有 2 个子节点。
- **满二叉树 (Full Binary Tree):** 每个节点有 0 或 2 个子节点。
- **完全二叉树 (Complete Binary Tree):** 除最后一层外，每层都完全填满，最后一层左对齐。

6.1.1 卡特兰数 (Catalan Numbers)

n 个节点能构成的不同二叉树的个数由卡特兰数给出：

$$C_n = \frac{1}{n+1} \binom{2n}{n}$$

$C_0, C_1, C_2, C_3, C_4, C_5 = 1, 1, 2, 5, 14, 42$ 。

6.2 二叉树

```
1 class TreeNode:
2     """ 二叉树节点。 """
3     def __init__(self, val=0, left=None, right=None):
4         self.val = val
5         self.left = left
6         self.right = right
```

6.3 树的遍历

6.3.1 前序遍历 (根 → 左 → 右)

前序遍历等价于标准 DFS 访问顺序，用于复制树和前缀表达式。

```
1 def preorderTraversal(root):
2     """ 二叉树的前序遍历 (递归)。 """
3     if not root:
4         return
5     print(root.val)
6     preorderTraversal(root.left)
7     preorderTraversal(root.right)
8
9 def preorderIterative(root):
10    """ 二叉树的前序遍历 (迭代)。 """
11    if not root:
12        return []
13    res, stack = [], [root]
14    while stack:
15        node = stack.pop()
16        res.append(node.val)
17        if node.right:
18            stack.append(node.right)
19        if node.left:
20            stack.append(node.left)
21    return res
```

6.3.2 中序遍历 (左 → 根 → 右)

对于二叉搜索树 (BST)，中序遍历得到升序序列。

```
1 def inorderTraversal(root):
2     """ 二叉树的中序遍历 (递归)。 """
```

```

3     res = []
4     def dfs(node):
5         if not node:
6             return
7         dfs(node.left)
8         res.append(node.val)
9         dfs(node.right)
10    dfs(root)
11    return res
12
13    def inorderIterative(root):
14        """ 二叉树的中序遍历 (迭代)。 """
15        res, stack = [], []
16        cur = root
17        while stack or cur:
18            while cur:
19                stack.append(cur)
20                cur = cur.left
21            cur = stack.pop()
22            res.append(cur.val)
23            cur = cur.right
24        return res

```

6.3.3 后序遍历 (左 → 右 → 根)

用于删除树和计算后缀表达式。关键观察：

$$\text{rev(后序遍历)} = \text{右子树优先的 DFS}$$

```

1    def postorderTraversal(root):
2        """ 二叉树的后序遍历 (递归)。 """
3        if not root:
4            return
5        postorderTraversal(root.left)
6        postorderTraversal(root.right)
7        print(root.val)

```

6.3.4 层序遍历 (BFS)

```

1    from collections import deque
2
3    def levelOrder(root):
4        """ 二叉树的层序遍历。 """
5        if not root:
6            return []
7        res, q = [], deque([root])

```

```

8     while q:
9         level = []
10        for _ in range(len(q)):
11            node = q.popleft()
12            level.append(node.val)
13            if node.left:
14                q.append(node.left)
15            if node.right:
16                q.append(node.right)
17        res.append(level)
18    return res

```

练习题目

- 二叉树的最大深度 (E104): $1 + \max(\text{depth}(\text{left}), \text{depth}(\text{right}))$ 。
- 翻转二叉树 (E226): 递归交换左右子树。
- 对称二叉树 (E101): 检查左子树是否是右子树的镜像。
- 从前序与中序遍历序列构造二叉树 (M105): 前序首元素为根, 在中序中定位以划分子树。
- 二叉树中的最大路径和 (T124): 后序遍历, 跟踪全局最大值。
- 二叉树的最近公共祖先 (M236): 递归查找; 若左和右均找到目标, 当前节点即 LCA。
- 二叉树的直径 (E543): 后序遍历, 对每个节点计算左右子树深度和, 取最大值。

6.4 二叉搜索树 (BST)

对于每个节点: 左子树所有值 < 节点值 < 右子树所有值。

```

1 class BST:
2     """ 二叉搜索树。 """
3     def __init__(self):
4         self.root = None
5
6     def search(self, val):
7         """ 查找值是否存在。 """
8         cur = self.root

```

```
9         while cur:
10             if val == cur.val:
11                 return True
12             cur = cur.left if val < cur.val else cur.right
13         return False
14
15     def insert(self, val):
16         """ 插入新值。 """
17         if not self.root:
18             self.root = TreeNode(val)
19             return
20         cur = self.root
21         while True:
22             if val < cur.val:
23                 if cur.left:
24                     cur = cur.left
25                 else:
26                     cur.left = TreeNode(val)
27                     return
28             else:
29                 if cur.right:
30                     cur = cur.right
31                 else:
32                     cur.right = TreeNode(val)
33                     return
34
35     def delete(self, val):
36         """ 删除值为 val 的节点。 """
37         def successor(node):
38             """ 寻找中序后继节点（右子树的最左下节点）。 """
39             node = node.right
40             while node.left:
41                 node = node.left
42             return node.val
43
44         def _delete(node, val):
45             if not node:
46                 return None
47             if val < node.val:
48                 node.left = _delete(node.left, val)
49             elif val > node.val:
50                 node.right = _delete(node.right, val)
51             else:
52                 if not node.left:
53                     return node.right
54                 if not node.right:
55                     return node.left
56                 node.val = successor(node)
57                 node.right = _delete(node.right, node.val)
58             return node
59
```

```
60     self.root = _delete(self.root, val)
```

复杂度: $O(h)$, 其中 h 为树高。平衡 BST: $h = O(\log n)$ 。退化为链表: $h = O(n)$ 。

例题: 验证二叉搜索树 (LeetCode 98)

```
1 def isValidBST(root):
2     """ 验证是否为有效的二叉搜索树。 """
3     def valid(node, lo, hi):
4         if not node:
5             return True
6         if not (lo < node.val < hi):
7             return False
8         return valid(node.left, lo, node.val) and \
9             valid(node.right, node.val, hi)
10    return valid(root, float('-inf'), float('inf'))
```

例题: BST 中第 K 小的元素 (LeetCode 230)

BST 的中序遍历产生有序序列; 遍历到第 k 个元素即可:

```
1 def kthSmallest(root, k):
2     """ 返回二叉搜索树中第 k 小的元素值。 """
3     stack = []
4     while stack or root:
5         while root:
6             stack.append(root)
7             root = root.left
8         root = stack.pop()
9         k -= 1
10        if k == 0:
11            return root.val
12        root = root.right
```

6.5 堆 (Heap)

堆是一个完全二叉树, 满足堆性质:

- 最小堆: 父节点 $\leq$ 子节点, 根为最小值。
- 最大堆: 父节点 $\geq$ 子节点, 根为最大值。

数组表示 (1 索引): 左子节点在 $2i$, 右子节点在 $2i+1$, 父节点在 $\lfloor i/2 \rfloor$ 。

```

1  import heapq
2
3  # 最小堆操作
4  heap = []
5  heapq.heappush(heap, 5)      #  $O(\log n)$ 
6  heapq.heappush(heap, 3)
7  min_val = heapq.heappop(heap) #  $O(\log n)$ , 返回 3
8
9  # 最大堆: 取反值即可
10 max_heap = []
11 heapq.heappush(max_heap, -5)
12 max_val = -heapq.heappop(max_heap) # 5
13
14 # 从列表构建堆:  $O(n)$ 
15 nums = [4, 1, 7, 3, 8, 5]
16 heapq.heapify(nums)

```

例题: 合并 K 个升序链表 (LeetCode 23)

```

1  import heapq
2
3  def mergeKLists(lists):
4      """ 使用最小堆合并 k 个有序链表。 """
5      dummy = cur = ListNode(0)
6      heap = []
7      for i, node in enumerate(lists):
8          if node:
9              heapq.heappush(heap, (node.val, i, node))
10     while heap:
11         _, i, node = heapq.heappop(heap)
12         cur.next = node
13         cur = cur.next
14         if node.next:
15             heapq.heappush(heap, (node.next.val, i, node.next))
16     return dummy.next

```

6.6 哈夫曼编码 (Huffman Coding)

哈夫曼编码是一种贪心无损压缩算法。出现频率越高的字符得到越短的编码。

6.6.1 算法步骤

1. 根据字符频率构建最小堆。
2. 反复取出两个频率最小的节点；创建一个内部节点合并它们的频率。
3. 最终的树定义编码：左边界 = 0，右边界 = 1。

```
1  import heapq
2  from collections import Counter
3
4  class HuffmanNode:
5      """ 哈夫曼树节点。 """
6      def __init__(self, char, freq):
7          self.char, self.freq = char, freq
8          self.left = self.right = None
9      def __lt__(self, other):
10         return self.freq < other.freq
11
12 def buildHuffmanTree(text):
13     """ 从文本构建哈夫曼树。 """
14     heap = [HuffmanNode(ch, f) for ch, f in Counter(text).items()]
15     heapq.heapify(heap)
16     while len(heap) > 1:
17         l, r = heapq.heappop(heap), heapq.heappop(heap)
18         p = HuffmanNode(None, l.freq + r.freq)
19         p.left, p.right = l, r
20         heapq.heappush(heap, p)
21     return heap[0]
22
23 def generateCodes(root, prefix="", codebook=None):
24     """ 从哈夫曼树生成编码表。 """
25     if codebook is None:
26         codebook = {}
27     if root.char is not None:
28         codebook[root.char] = prefix
29         return codebook
30     if root.left:
31         generateCodes(root.left, prefix + "0", codebook)
32     if root.right:
33         generateCodes(root.right, prefix + "1", codebook)
34     return codebook
```

练习：哈夫曼编码树 (OpenJudge 04080)

给定字符频率构建哈夫曼树，计算带权路径长度 (WPL)。

6.7 字典树 (Trie)

Trie 将字符串存储为从根到叶子的路径。共享公共前缀的字符串复用路径——是自动补全和字典查询的理想选择。

```
1 class TrieNode:
2     """Trie 节点。"""
3     def __init__(self):
4         self.children = {}
5         self.is_end = False
6
7 class Trie:
8     """前缀树 / 字典树。"""
9     def __init__(self):
10         self.root = TrieNode()
11
12     def insert(self, word: str) -> None:
13         """插入单词。"""
14         node = self.root
15         for ch in word:
16             if ch not in node.children:
17                 node.children[ch] = TrieNode()
18             node = node.children[ch]
19         node.is_end = True
20
21     def search(self, word: str) -> bool:
22         """搜索单词是否存在。"""
23         node = self.root
24         for ch in word:
25             if ch not in node.children:
26                 return False
27             node = node.children[ch]
28         return node.is_end
29
30     def startsWith(self, prefix: str) -> bool:
31         """检查是否有以 prefix 为前缀的单词。"""
32         node = self.root
33         for ch in prefix:
34             if ch not in node.children:
35                 return False
36             node = node.children[ch]
37         return True
```

练习：最长公共后缀查询 (LeetCode 3093)

将字符串反转后存入 Trie（后缀 Trie）以支持高效的后缀查询。

6.8 并查集 (Disjoint Set Union)

并查集管理元素的不相交集合划分。配合路径压缩和按秩合并，操作几乎为 $O(1)$ 。

```
1 class DSU:
2     """ 并查集 ——含路径压缩与按秩合并。 """
3     def __init__(self, n):
4         self.parent = list(range(n))
5         self.rank = [1] * n
6
7     def find(self, x):
8         """ 查找根节点，含路径压缩。 """
9         if self.parent[x] != x:
10             self.parent[x] = self.find(self.parent[x])
11         return self.parent[x]
12
13     def union(self, x, y):
14         """ 合并两个集合，按秩合并。返回是否成功合并。 """
15         rx, ry = self.find(x), self.find(y)
16         if rx == ry:
17             return False
18         if self.rank[rx] < self.rank[ry]:
19             rx, ry = ry, rx
20         self.parent[ry] = rx
21         if self.rank[rx] == self.rank[ry]:
22             self.rank[rx] += 1
23         return True
```

练习：The Suspects (OpenJudge 01611)、虫子的生活 (OpenJudge 07734)

并查集天然适用于建模“连通”或“属于同一组”的关系。对于二分图冲突等问题，可使用带权并查集（也称“食物链并查集”）。

Chapter 7

图

图用于建模实体之间的关系。树和链表都是图的特例。

7.1 术语与定义

- 顶点 (Vertex, V): 图的基本单元。
- 边 (Edge, E): 连接两个顶点的连线。
- 有向图 vs. 无向图: 边是否有方向性。
- 加权图: 边带有权重 (成本)。
- 路径 (Path): 由边连接的顶点序列。
- 环 (Cycle): 起点与终点相同的路径。
- DAG: 有向无环图。
- 度 (Degree): 无向图中与顶点关联的边数。有向图中分为入度和出度。

7.2 图的表示

7.2.1 邻接矩阵

$n \times n$ 矩阵: 若边 (i, j) 存在, 则 $A[i][j] = 1$ (或权重)。

- 空间： $O(V^2)$ 。边查询： $O(1)$ 。找邻居： $O(V)$ 。
- 适用：稠密图。

7.2.2 邻接表

每个顶点维护邻接列表（或字典）：

- 空间： $O(V + E)$ 。找邻居： $O(\text{degree}(v))$ 。
- 适用：稀疏图（大多数实际场景）。

```

1  from collections import defaultdict, deque
2
3  class Graph:
4      """ 使用邻接表表示的图。 """
5      def __init__(self):
6          self.adj = defaultdict(list)
7
8      def addEdge(self, u, v, directed=False):
9          """ 添加边。若为无向边则同时添加反向边。 """
10         self.adj[u].append(v)
11         if not directed:
12             self.adj[v].append(u)

```

7.3 图的遍历

7.3.1 BFS（广度优先搜索）

BFS 使用队列按层探索。能够找到无权图中的最短路径。

```

1  def bfs(graph, start):
2      """ 图的广度优先搜索。 """
3      visited = {start}
4      q = deque([start])
5      order = []
6      while q:
7          u = q.popleft()
8          order.append(u)
9          for v in graph[u]:
10             if v not in visited:
11                 visited.add(v)
12                 q.append(v)
13      return order

```

7.3.2 DFS（深度优先搜索）

DFS 沿每条分支尽可能深入，然后回溯。

```
1 def dfs(graph, start):
2     """ 图的深度优先搜索（递归）。"""
3     visited = set()
4     order = []
5     def _dfs(u):
6         visited.add(u)
7         order.append(u)
8         for v in graph[u]:
9             if v not in visited:
10                 _dfs(v)
11     _dfs(start)
12     return order
13
14 def dfsIterative(graph, start):
15     """ 图的深度优先搜索（迭代）。"""
16     visited, stack = {start}, [start]
17     order = []
18     while stack:
19         u = stack.pop()
20         order.append(u)
21         for v in graph[u]:
22             if v not in visited:
23                 visited.add(v)
24                 stack.append(v)
25     return order
```

例题：岛屿数量 (LeetCode 200)

将二维网格当作隐式图，用 DFS 洪泛填充相连的'1'：

```
1 def numIslands(grid):
2     """ 计算网格中岛屿的数量。"""
3     m, n = len(grid), len(grid[0])
4     def dfs(i, j):
5         if i < 0 or i >= m or j < 0 or j >= n or grid[i][j] != '1':
6             return
7         grid[i][j] = '0'
8         for di, dj in ((1,0), (-1,0), (0,1), (0,-1)):
9             dfs(i + di, j + dj)
10
11     count = 0
12     for i in range(m):
13         for j in range(n):
14             if grid[i][j] == '1':
```

```

15         count += 1
16         dfs(i, j)
17     return count

```

练习：扫雷游戏 (LeetCode 529)、Kefa and Park (CF 580C)

7.4 拓扑排序

拓扑排序是 DAG 中所有顶点的线性排序，使每条有向边 $u \rightarrow v$ 中 u 排在 v 之前。

7.4.1 Kahn 算法 (BFS 实现)

```

1 def topologicalSort(n, edges):
2     """Kahn 算法求拓扑序。
3     参数:
4         n: 顶点数。
5         edges: 列表，每项为 (u, v)，表示有向边 u -> v。
6     返回:
7         拓扑序列，若存在环则返回空列表。
8     """
9     graph = defaultdict(list)
10    indegree = [0] * n
11    for u, v in edges:
12        graph[u].append(v)
13        indegree[v] += 1
14
15    q = deque([i for i in range(n) if indegree[i] == 0])
16    order = []
17    while q:
18        u = q.popleft()
19        order.append(u)
20        for v in graph[u]:
21            indegree[v] -= 1
22            if indegree[v] == 0:
23                q.append(v)
24    return order if len(order) == n else [] # 有环则返回空

```

例题：课程表 (LeetCode 207)

判断能否完成所有课程——只需检查是否存在拓扑序（无环）：

```

1 def canFinish(numCourses, prerequisites):

```

```

2     """ 判断是否能完成所有课程。"""
3     return len(topologicalSort(numCourses, prerequisites)) == numCourses

```

7.5 AOE 网与关键路径

7.5.1 AOE 网 (Activity On Edge)

AOE 网是一种带权的有向无环图，用于工程调度：

- **顶点**：表示事件（某个任务完成的时刻）。
- **有向边**：表示活动（从一事件到另一事件的过程），边权重为该活动的持续时间。
- **源点**：入度为 0 的顶点（工程开始）。
- **汇点**：出度为 0 的顶点（工程完成）。

7.5.2 关键路径 (Critical Path)

关键路径是 AOE 网中从源点到汇点的**最长路径**。它决定了工程的最早完成时间。关键路径上的活动不允许延迟，否则将推迟整个工程。

7.5.3 核心量

- $ve(i)$ ：事件 i 的**最早发生时间**——从源点到 i 的最长路径长度。
- $vl(i)$ ：事件 i 的**最迟发生时间**——不推迟工程的前提下，事件允许的最晚时间。
- $ee(a)$ ：活动 a 的**最早开始时间**， $ee(a) = ve(\text{tail}(a))$ 。
- $el(a)$ ：活动 a 的**最迟开始时间**， $el(a) = vl(\text{head}(a)) - w(a)$ 。
- **时间余量**： $el(a) - ee(a)$ 。若为 0，则该活动在**关键路径**上。

7.5.4 算法步骤

1. 按拓扑序正向计算 ve : $ve[v] = \max\{ve[u] + w(u, v)\}$ 。
2. 按拓扑逆序反向计算 vl : $vl[u] = \min\{vl[v] - w(u, v)\}$ 。
3. 若 $ve[i] = vl[i]$, 则顶点 i 在关键路径上。若 $ee(a) = el(a)$, 则活动 a 是关键活动。

```

1  def criticalPath(n, edges):
2      """ 计算 AOE 网的关键路径长度与关键活动。
3      参数:
4          n: 顶点数。
5          edges: (u, v, w) 列表, 表示活动从 u 到 v, 持续时间为 w。
6      返回:
7          (工程最短完成时间, 关键活动列表)。
8      """
9      graph = defaultdict(list)
10     indegree = [0] * n
11     for u, v, w in edges:
12         graph[u].append((v, w))
13         indegree[v] += 1
14
15     # 第 1 步: 拓扑排序
16     q = deque([i for i in range(n) if indegree[i] == 0])
17     topo = []
18     while q:
19         u = q.popleft()
20         topo.append(u)
21         for v, w in graph[u]:
22             indegree[v] -= 1
23             if indegree[v] == 0:
24                 q.append(v)
25
26     # 第 2 步: 正向计算 ve (最早发生时间)
27     ve = [0] * n
28     for u in topo:
29         for v, w in graph[u]:
30             ve[v] = max(ve[v], ve[u] + w)
31
32     # 第 3 步: 反向计算 vl (最迟发生时间)
33     vl = [ve[-1]] * n # 初始化为汇点的 ve
34     for u in reversed(topo):
35         for v, w in graph[u]:
36             vl[u] = min(vl[u], vl[v] - w)
37
38     # 第 4 步: 找出关键活动
39     critical = []
40     for u in range(n):

```

```

41     for v, w in graph[u]:
42         if ve[u] == vl[u] and ve[v] == vl[v] and ve[u] + w == ve[v]:
43             # 活动 u->v 在关键路径上
44             critical.append((u, v, w))
45
46     return ve[-1], critical

```

复杂度: $O(V + E)$ 。关键路径方法广泛应用于工程项目管理 (PERT 图)、制造调度和资源优化。

7.6 最短路径

7.6.1 Dijkstra 算法

Dijkstra 算法在非负权图中找到单源最短路径, 使用最小堆优先队列。

```

1  import heapq
2
3  def dijkstra(graph, start):
4      """Dijkstra 单源最短路径。
5      参数:
6          graph: 邻接表 {u: [(v, weight), ...]}。
7          start: 源顶点。
8      返回:
9          {顶点: 最短距离} 字典。
10     """
11     dist = {start: 0}
12     pq = [(0, start)]
13     while pq:
14         d, u = heapq.heappop(pq)
15         if d > dist.get(u, float('inf')):
16             continue # 跳过过期条目
17         for v, w in graph.get(u, []):
18             nd = d + w
19             if nd < dist.get(v, float('inf')):
20                 dist[v] = nd
21                 heapq.heappush(pq, (nd, v))
22     return dist

```

复杂度: $O((V + E) \log V)$ 。限制: 不能处理负权边。

练习：兔子与樱花 (OpenJudge 05443)、地铁换乘 (OpenJudge 2502)

7.6.2 Bellman-Ford 算法

Bellman-Ford 可以处理负权边并检测负环：

```

1 def bellmanFord(n, edges, start):
2     """Bellman-Ford 算法（支持负权边，检测负环）。
3     参数：
4         n: 顶点数。
5         edges: (起点, 终点, 权重) 的列表。
6         start: 源顶点。
7     返回：
8         (距离数组, 是否存在负环)。
9     """
10    dist = [float('inf')] * n
11    dist[start] = 0
12    for _ in range(n - 1):
13        updated = False
14        for u, v, w in edges:
15            if dist[u] != float('inf') and dist[u] + w < dist[v]:
16                dist[v] = dist[u] + w
17                updated = True
18        if not updated:
19            break # 提前终止
20    # 检测负环
21    for u, v, w in edges:
22        if dist[u] != float('inf') and dist[u] + w < dist[v]:
23            return dist, True # 存在负环
24    return dist, False

```

复杂度： $O(VE)$ 。

7.6.3 Floyd-Warshall 算法

使用 DP 计算所有顶点对的最短路径，时间复杂度 $O(V^3)$ ：

```

1 def floydWarshall(n, edges):
2     """Floyd-Warshall 全源最短路径。
3     参数：
4         n: 顶点数。
5         edges: (u, v, weight) 列表。
6     返回：
7         n x n 的距离矩阵。
8     """

```

```

9     INF = float('inf')
10    dist = [[INF] * n for _ in range(n)]
11    for i in range(n):
12        dist[i][i] = 0
13    for u, v, w in edges:
14        dist[u][v] = min(dist[u][v], w)
15
16    for k in range(n):
17        for i in range(n):
18            for j in range(n):
19                if dist[i][k] != INF and dist[k][j] != INF:
20                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
21    return dist

```

例题：同余最短路 (洛谷 P3403)

在模图（同余最短路）上使用最短路径框架——每个顶点表示一个余数类，边表示加一个数。

7.7 最小生成树 (MST)

MST 是连通无向加权图的一个生成树，其边权总和最小。

7.7.1 Kruskal 算法

将边按权重排序；贪心地添加不产生环的边（通过并查集检查）：

```

1  def kruskal(n, edges):
2      """Kruskal 最小生成树算法。
3      参数：
4          n: 顶点数。
5          edges: (u, v, weight) 列表。
6      返回：
7          MST 中的边列表。
8      """
9      edges.sort(key=lambda x: x[2])
10     dsu = DSU(n)
11     mst = []
12     for u, v, w in edges:
13         if dsu.union(u, v):
14             mst.append((u, v, w))
15         if len(mst) == n - 1:
16             break
17     return mst

```

复杂度: $O(E \log E)$ 。

练习: Agri-Net (OpenJudge 01258)、兔子与星空 (OpenJudge 05442)

7.7.2 Prim 算法

Prim 算法从起始顶点出发扩展 MST，每次选择连接树与新顶点的最便宜边：

```

1  import heapq
2
3  def prim(graph, n):
4      """Prim 最小生成树算法。
5      参数:
6          graph: 邻接表 {u: [(v, weight), ...]}。
7          n: 顶点数。
8      返回:
9          MST 中的边列表。
10     """
11     visited = [False] * n
12     mst = []
13     pq = [(0, -1, 0)] # (权重, 起点, 终点), 从顶点 0 开始
14     while pq and len(mst) < n - 1:
15         w, u, v = heapq.heappop(pq)
16         if visited[v]:
17             continue
18         visited[v] = True
19         if u != -1:
20             mst.append((u, v, w))
21         for nxt, nw in graph.get(v, []):
22             if not visited[nxt]:
23                 heapq.heappush(pq, (nw, v, nxt))
24     return mst

```

复杂度: $O((V + E) \log V)$ 。

7.8 强连通分量 (SCC)

有向图中，一个强连通分量 (SCC) 是极大的顶点集合，其中任意两个顶点相互可达。

7.8.1 Tarjan 算法

Tarjan 算法通过一次 DFS 即可找到所有 SCC，过程中维护两个数组：

- `dfn[u]`：DFS 首次访问顶点 u 的时间戳。
- `low[u]`：从 u 出发，通过一系列树边和最多一条回边（或横叉边）能到达的最早顶点的 `dfn` 值。

当 `dfn[u] = low[u]` 时，说明 u 是某个 SCC 的根——此时栈中 u 及其上方的所有顶点构成一个 SCC。

```

1 def tarjan(n, edges):
2     """Tarjan 算法求有向图的强连通分量。
3     参数：
4         n: 顶点数。
5         edges: (u, v) 列表，表示有向边 u -> v。
6     返回：
7         SCC 列表，每个 SCC 为顶点列表。
8     """
9     graph = defaultdict(list)
10    for u, v in edges:
11        graph[u].append(v)
12
13    dfn = [0] * n        # DFS 访问时间戳，0 表示未访问
14    low = [0] * n        # 从当前节点能回溯到的最早节点的时间戳
15    in_stack = [False] * n
16    stack = []
17    sccs = []
18    time = 0
19
20    def dfs(u):
21        nonlocal time
22        time += 1
23        dfn[u] = low[u] = time
24        stack.append(u)
25        in_stack[u] = True
26
27        for v in graph[u]:
28            if dfn[v] == 0:        # v 未被访问，是树边
29                dfs(v)
30                low[u] = min(low[u], low[v])
31            elif in_stack[v]:        # v 在栈中，是回边或横叉边
32                low[u] = min(low[u], dfn[v])
33
34        # 如果 u 是 SCC 的根
35        if dfn[u] == low[u]:
36            scc = []

```



```
37         while True:
38             v = stack.pop()
39             in_stack[v] = False
40             scc.append(v)
41             if v == u:
42                 break
43             sccs.append(scc)
44
45     for u in range(n):
46         if dfn[u] == 0:
47             dfs(u)
48
49     return sccs
```

复杂度: $O(V + E)$ 。Tarjan 算法仅需一次 DFS，是求解 SCC 的最高效方法之一。

练习: 间谍网络 (洛谷 P1262)、Checkposts (CF 427C)、缩点 (洛谷 P3387)

SCC 缩点（将每个 SCC 缩为一个顶点）可将有向图转换为 DAG，便于进一步处理（如在 DAG 上做 DP、拓扑排序）。

附录 A

常用算法模式

A.1 双指针

```
1 def maxArea(height):
2     """ 盛最多水的容器 (M11)。"""
3     l, r, ans = 0, len(height) - 1, 0
4     while l < r:
5         ans = max(ans, min(height[l], height[r]) * (r - l))
6         if height[l] < height[r]:
7             l += 1
8         else:
9             r -= 1
10    return ans
11
12 def trap(height):
13     """ 接雨水 (T42)。"""
14     l, r = 0, len(height) - 1
15     l_max = r_max = ans = 0
16     while l < r:
17         if height[l] < height[r]:
18             if height[l] >= l_max:
19                 l_max = height[l]
20             else:
21                 ans += l_max - height[l]
22             l += 1
23         else:
24             if height[r] >= r_max:
25                 r_max = height[r]
26             else:
27                 ans += r_max - height[r]
28             r -= 1
29    return ans
```

```

30
31 def threeSum(nums):
32     """ 三数之和 (M15)。 """
33     nums.sort()
34     res = []
35     for i in range(len(nums) - 2):
36         if i > 0 and nums[i] == nums[i - 1]:
37             continue
38         l, r = i + 1, len(nums) - 1
39         while l < r:
40             s = nums[i] + nums[l] + nums[r]
41             if s < 0:
42                 l += 1
43             elif s > 0:
44                 r -= 1
45             else:
46                 res.append([nums[i], nums[l], nums[r]])
47                 while l < r and nums[l] == nums[l + 1]:
48                     l += 1
49                 while l < r and nums[r] == nums[r - 1]:
50                     r -= 1
51                 l, r = l + 1, r - 1
52     return res

```

A.2 滑动窗口

```

1 def lengthOfLongestSubstring(s):
2     """ 无重复字符的最长子串 (M3)。 """
3     seen, l, ans = set(), 0, 0
4     for r in range(len(s)):
5         while s[r] in seen:
6             seen.remove(s[l])
7             l += 1
8         seen.add(s[r])
9         ans = max(ans, r - l + 1)
10    return ans
11
12 def minWindow(s, t):
13     """ 最小覆盖子串 (T76)。 """
14     from collections import Counter
15     need = Counter(t)
16     missing = len(t)
17     l = start = end = 0
18     for r, ch in enumerate(s, 1):
19         if need[ch] > 0:
20             missing -= 1
21         need[ch] -= 1
22         if missing == 0:

```

```

23         while l < r and need[s[l]] < 0:
24             need[s[l]] += 1
25             l += 1
26         if end == 0 or r - l < end - start:
27             start, end = l, r
28             need[s[l]] += 1
29             missing += 1
30             l += 1
31     return s[start:end]

```

A.3 动态规划

```

1  def maxSubArray(nums):
2      """ 最大子数组和 (M53) — Kadane 算法。 """
3      cur = best = nums[0]
4      for num in nums[1:]:
5          cur = max(num, cur + num)
6          best = max(best, cur)
7      return best
8
9  def longestPalindrome(s):
10     """ 最长回文子串 (M5) — 中心扩展法。 """
11     n = len(s)
12     start, max_len = 0, 1
13     for i in range(n):
14         for l, r in ((i, i), (i, i + 1)):
15             while l >= 0 and r < n and s[l] == s[r]:
16                 if r - l + 1 > max_len:
17                     start, max_len = l, r - l + 1
18                 l -= 1
19                 r += 1
20     return s[start:start + max_len]

```

A.4 回溯

```

1  def permute(nums):
2      """ 全排列 (M46)。 """
3      res = []
4      def backtrack(path, used):
5          if len(path) == len(nums):
6              res.append(path[:])
7              return
8          for i in range(len(nums)):
9              if used[i]:
10                 continue

```

```

11         used[i] = True
12         path.append(nums[i])
13         backtrack(path, used)
14         path.pop()
15         used[i] = False
16     backtrack([], [False] * len(nums))
17     return res
18
19 def subsets(nums):
20     """ 子集 (M78)。 """
21     res = []
22     def backtrack(start, path):
23         res.append(path[:])
24         for i in range(start, len(nums)):
25             path.append(nums[i])
26             backtrack(i + 1, path)
27             path.pop()
28     backtrack(0, [])
29     return res

```

A.5 二分查找

```

1 def binarySearch(nums, target):
2     """ 在有序数组中二分查找目标值。 """
3     lo, hi = 0, len(nums) - 1
4     while lo <= hi:
5         mid = (lo + hi) // 2
6         if nums[mid] == target:
7             return mid
8         elif nums[mid] < target:
9             lo = mid + 1
10        else:
11            hi = mid - 1
12    return -1
13
14 def lowerBound(nums, target):
15     """ 第一个满足 nums[i] >= target 的索引 (下界)。 """
16     lo, hi = 0, len(nums)
17     while lo < hi:
18         mid = (lo + hi) // 2
19         if nums[mid] < target:
20             lo = mid + 1
21        else:
22            hi = mid
23    return lo
24
25 def searchMatrix(matrix, target):
26     """ 搜索二维矩阵 II (M240) — 从右上角开始排除。 """

```

```
27     if not matrix:
28         return False
29     i, j = 0, len(matrix[0]) - 1
30     while i < len(matrix) and j >= 0:
31         if matrix[i][j] == target:
32             return True
33         elif matrix[i][j] > target:
34             j -= 1
35         else:
36             i += 1
37     return False
```
